Real-time CTR prediction with online feature selection - Project report

Hubert Jaczyński, Aleksandra Kłos, Jakub Oganowski

29th April 2026

1 Introduction

1.1 Motivation

The modern digital advertising market is based on real-time bidding (RTB)[1]. In this type of system, the final impression value, which is measured by the eCPM metric[2], is dependent on the estimation that the user will interact with a given advertisement. Therefore, the precise estimation of click-through rate (CTR)[3] serves as an economic basis of the whole industry. In 2013, Google engineers wrote a paper presenting a selection of case studies and topics drawn from recent experiments in the setting of a deployed CTR prediction system and describing the architecture of the FTRL-Proximal model [1]. Moreover, they have pointed out that, on a global scale, even a minor improvement in prediction accuracy results in multimillion-dollar savings and a significant increase in content relevance for end users.

Nevertheless, this challenge is rather complex from both computational and analytical viewpoints. This is because RTB systems are obliged to process hundreds of thousands of queries per second under very rigorous time constraints. Moreover, the incoming ad logs have the character of an endless stream and are characterized by two problematic features: significant sparsity, as the vast majority of impressions do not end with a click, as well as considerably high dimensionality of data [2]. It stems from the fact that most signals are categorical variables with millions of unique values, such as unique IP addresses, device identifiers, and so forth.

Common machine learning methods using batch streaming are impractical in such circumstances. On the other hand, constant retraining of powerful models from scratch using a growing database is too expensive in terms of time and infrastructure. This requires the use of incremental learning algorithms. They can update their weights and structure on the fly, processing data instance by instance, immediately after receiving a notification of a click or its absence.

[1] RTB House, *Real-time bidding*, `https://www.rtbhouse.com/martech-terminology/real-time-bidding`.

[2] Unity, *What does eCPM stand for?*, `https://unity.com/glossary/ecpm`.

[3] Google Ads, *Clickthrough rate (CTR): Definition*, `https://support.google.com/google-ads/answer/2615875?hl=en`.

1.2 Research problem

The main issue in analyzing advertising data streams is their inherent mobility. Machine learning models are trained under the assumption of a constant data distribution, but in reality, this distribution changes over time. It is known as concept drift [3]. This change can affect both user preferences, such as seasonal purchasing patterns, and the effectiveness of advertising itself.

Besides, the phenomenon of feature drift, described in the literature on online feature selection [4], is especially applicable. It means that the set of features relevant to the model changes over time. Some parameters, for instance, the user's phone model, may suddenly become more important or become irrelevant altogether. As presented in the lecture, algorithms, such as decision trees, often fail to keep pace with these changes. Thus, they bring about the adaptation gap.

1.3 Research objective

Based on the observations described, our project aims to develop a state-of-the-art model capable of rapid adaptation in a non-stationary environment. The streaming random patches (SRP) algorithm [5] has been chosen to be the foundation. We have decided to choose it as it is currently one of the most effective ensemble solutions for data streams. Furthermore, the literature states that SRP is characterized by the use of background models that begin learning a new concept immediately after the ADWIN detector detects a drift.

Yet, our project's innovation extends the SRP mechanism with dynamic feature selection, described as online feature selection (OFS) [6]. Instead of relying on random subspaces, our algorithm will evaluate each feature's usefulness on the fly. This may show that when drift is detected, newly created ensemble components, background learners, will receive access to the most recent and informative parameters. They eventually focus on shortening recovery time and reducing logarithmic loss [7].

2 Data preparation

For this project, we have prepared three datasets: two from real advertising systems and one from a synthetic environment used to analyze the adaptation gap.

2.1 Real-time data streams: Avazu and Criteo

RTB datasets are characterized by their vast volume and sparsity. Special care was taken to preserve the original chronological order of the rows. This is because random data shuffling could destroy the naturally occurring concept drift. Hence, the evaluation becomes impractical.

2.1.1 Exploratory data analysis – Avazu dataset

The dataset has been obtained from the Kaggle competition[4]. The competition data provides separate training and testing files. However, for the purpose of this EDA and

[4]Kaggle, *Click-Through Rate Prediction - Avazu*, `https://www.kaggle.com/competitions/avazu-ctr-prediction/`.

later for model evaluations, we have used `train.csv` data. Besides, to save RAM on our devices, while preserving the nature of streaming data, the first $1,000,000$ records has been analyzed.

The subset consists of 24 columns in total: 23 predictive features and 1 target variable (`click`). The vast majority of the features are categorical and include explicit attributes describing the user's hardware and site context, such as `device_id`, `device_ip`, `site_domain`, `app_category`, as well as anonymized categorical variables like `C1`, `C14`–`C21`.

Target variable and class imbalance The target feature, `click`, is a binary variable that indicates whether an advertisement was clicked (1) or not (0). The distribution analysis showed a serious class imbalance. It is a common characteristic of RTB systems. Specifically, only 16.02% of the impressions resulted in a click, whereas 83.98% were ignored. As seen in Figure 1, this highly skewed distribution ($P(Y = 0) \gg P(Y = 1)$) disqualifies the use of the traditional accuracy metric and encourages us to use the probabilistic logarithmic loss function for further model evaluation.

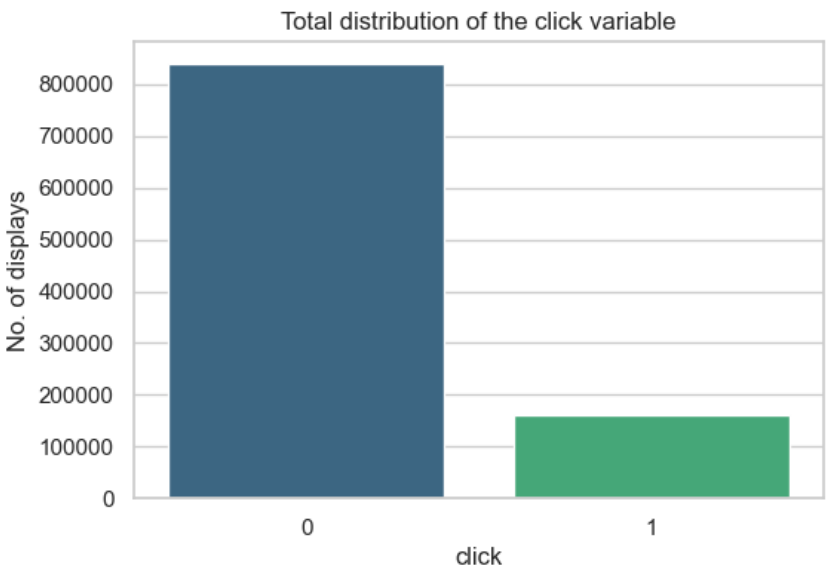


Figure 1: Total distribution of the `click` variable.

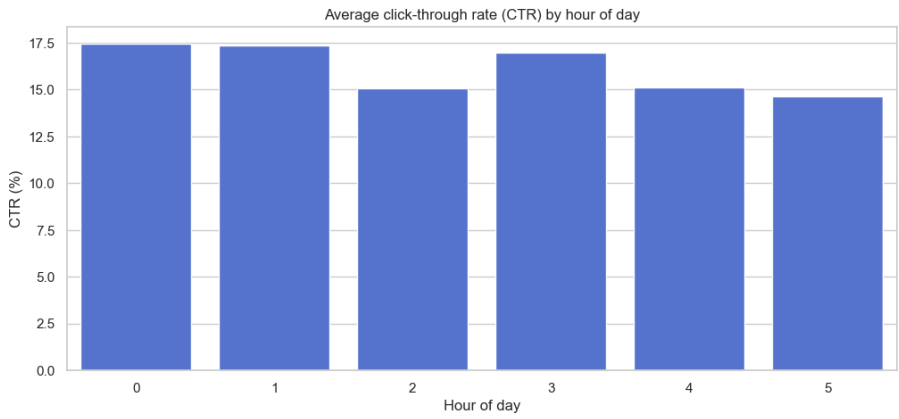
High cardinality of categorical features The major challenge of the Avazu dataset is the high cardinality of its categorical features. Excluding the unique `id` column, the analysis showed that single features contain up to hundreds of thousands of unique values. As seen in Table 1, within the $1,000,000$ rows sample, the `device_ip` column contains $313,002$ unique values, `device_id` contains $83,431$, and `device_model` contains $4,581$ unique categories. Such dimensionality makes one-hot encoding computationally infeasible, which justifies the use of hashing techniques.

Table 1: Feature cardinality of the first 5 features in the Avazu dataset.

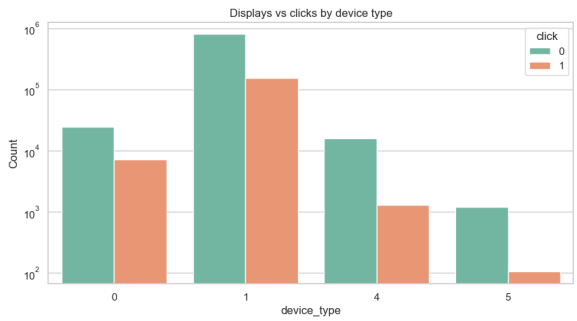
Feature name	No. of unique values
`id`	1,000,000
`device_ip`	313,002
`device_id`	83,431
`device_model`	4,581
`app_id`	2,309

CTR analysis Further analysis of the CTR has been conducted across different dimensions. The analysis made was as follows, as provided in Figure 2:

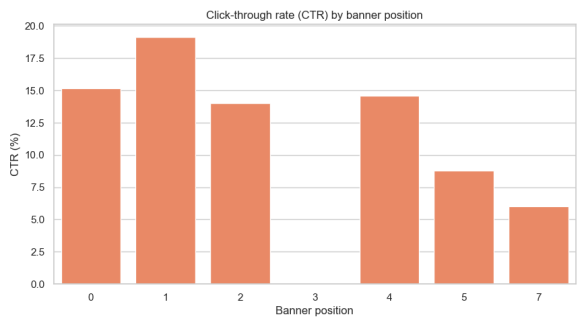
- **Temporal variability (plot A):** The average CTR is not constant but fluctuates depending on the hour of the day. It suggests the presence of concept drift in the stream.
- **Device type influence (plot B):** The distribution of displays and clicks varies by `device_type`. Even though one device dominates overall traffic, click rates vary by platform.
- **Advertisement placement (plot C):** The analysis of the `banner_pos` feature demonstrated a correlation between the physical placement of the banner on the screen and the resulting CTR.



(a) Average CTR by hour of day



(b) Displays vs clicks by device type



(c) CTR by banner position

Figure 2: CTR in Avazu dataset analysis.

2.1.2 Exploratory data analysis – Criteo dataset

The Criteo dataset is an online advertising dataset released by Criteo Labs[5]. It contains feature values and click feedback for millions of display advertisements and serves as a widely used benchmark for CTR prediction. As in the Avazu dataset, we used the `train.txt` file, and $1,000,000$ records were analyzed.

The dataset is heterogeneous and comprises 40 columns. The first attribute is the target variable (`label`), where 1 indicates the advertisement was clicked and 0 indicates it was not. The remaining 39 predictive attributes consist of 13 continuous numerical columns (`I1`–`I13`) and 26 anonymized categorical columns (`C1`–`C26`).

Target variable and class imbalance The target feature, `label`, is a binary variable representing user interactions. Once again, as presented in Figure 3, the distribution analysis of the sampled records revealed a notable class imbalance. Particularly, 25.49% of the impressions resulted in a click, while 74.51% were ignored. Although the CTR is slightly higher compared to the Avazu dataset, the data remains heavily skewed ($P(Y = 0) > P(Y = 1)$) and we cannot use the traditional accuracy metrics.

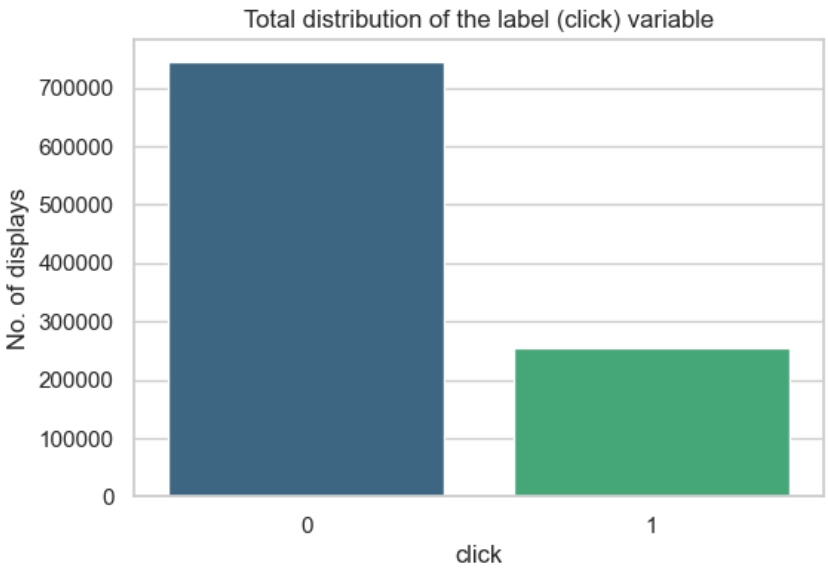


Figure 3: Total distribution of the `label` (click) variable.

Missing data and numerical skewness Unlike the Avazu dataset, the Criteo dataset poses the challenge of continuous variables. EDA showed that the numerical features (`I1`–`I13`) are sparsely populated, with varying percentages of missing values (NaNs) across columns. Furthermore, the existing continuous values exhibit highly right-skewed, long-tail distributions, as shown in Figure 4. They are populated by values close to zero but contain vivid outliers, as shown by truncating feature `I1` at the 99th percentile. The sparseness and skewness have required data preprocessing and an imputation strategy, such as filling missing numerical values with zeros, before model training.

High cardinality of categorical features The 26 categorical features (`C1`–`C26`) consist of 32-bit hashed strings. They exhibit a high cardinality similar to the Avazu dataset,

[5]Criteo AI Lab, *Criteo Research Datasets*, `https://ailab.criteo.com/ressources/`

despite the fact they have remained anonymized.

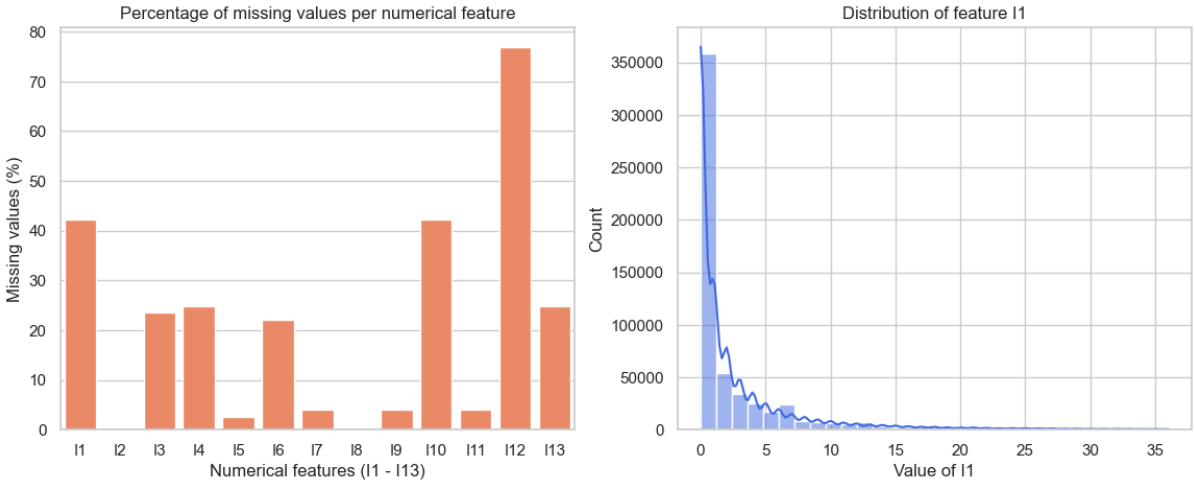


Figure 4: Criteo dataset analysis: (A) Percentage of missing values per numerical feature, (B) Skewed distribution of continuous feature `I1`.

2.2 Feature engineering and hashing

As demonstrated in EDA, data streams with high cardinality and missing values can easily overload an online classifier's memory. To solve this, we have built a code that creates new features, reduces dimensionality, and saves the final output as an `ARFF` file for the Massive Online Analysis (MOA) framework [8].

Temporal and moving window features To help the model adapt to recent trends and concept drift, we have created dynamic features:

- **Avazu:** We have extracted specific time details (hour, day, week) and created interaction features`site_x_hour`). We have also used a sliding window of 1000 instances to calculate the recent CTR and interaction frequencies for sites and apps.
- **Criteo:** We first replaced missing numeric values with zeros. Then, we have applied a sliding window of 1000 instances to the first five numerical columns (`I1`–`I5`) to compute their moving averages, standard deviations, and deviations from the mean.

Dimensionality reduction via feature hashing As mentioned in Section 2.1.1, techniques like one-hot encoding create too many columns for streaming data. Instead, we have decided to use the feature hashing algorithm [9] from *scikit-learn* library to compress all categorical strings into a fixed-size vector of exactly 100 columns. This has provided two main benefits:

1. The model's size stays the same, even if new, unseen categories appear in the stream.
2. By keeping the number of columns strictly bounded, the SRP ensemble can process data much more quickly.

Final state In the final step, we have merged all these parts together. For Avazu, we have combined the new time-based features with the 100 hashed columns. For Criteo on the other hand, we have combined the 13 original numeric columns, the 15 new moving statistics, and the 100 hashed columns. This optimized data has been then to train and evaluate our models.

2.3 Synthetic data stream: Agrawal generator

Real-world streams contain natural, hidden concept drifts. However, evaluating the exact reaction time of an algorithm requires a controlled environment. For this purpose, the Agrawal generator [10] has been used. This is a well-known synthetic data stream built directly into the MOA framework.

Stream characteristics The Agrawal generator simulates a loan approval process based on 9 attributes, such as age, salary and loan amount. Because the data is generated on the fly by mathematical functions within the MOA engine, it does not require a static ARFF file or memory-heavy preprocessing. In our experiment to keep the trend, we have generated a stream of exactly $1,000,000$ instances.

Abrupt concept drift injection To test the models' adaptability and measure the adaptation gap, we have designed a scenario with a sudden, severe change in the data distribution. We have achieved this by configuring the `ConceptDriftStream` within the MOA using three parameters:

- **Stream 1 (instances 1 – 500,000):** Configured as an `AgrawalGenerator` using classification function 1. During this phase, the models learned the initial rules and minimized their prediction error.
- **Drift position (instance 500,001):** The drift injection point was set exactly at the 500,000 mark. At this moment, the framework has injected an abrupt concept drift by switching the data generation logic.
- **Stream 2 (instances 500,001 – 1,000,000):** Configured with an Agrawal generator using classification function 2. Consequently, the old rules became obsolete, forcing the models to detect the change and learn the new concept quickly.

This setup has allowed us to measure the recovery time ($t_{recovery}$) after the drift.

2.4 Summary of data streams characteristics

Following the EDA, feature engineering, and dimensionality reduction phases, the three data streams were evaluated sequentially in the MOA framework. Table 2 presents the final characteristics of the processed streams.

Table 2: Summary of the final data streams characteristics used for model evaluation.

Dataset	Total instances	Final feature types	Feature count	Concept drift type
Avazu	1,000,000	Numeric + hashed categorical	109	Natural
Criteo	1,000,000	Continuous + hashed categorical	128	Natural
Agrawal	1,000,000	Continuous	9	Synthetic (abrupt at 500,000)

3 Evaluation methodology

3.1 Drift detection in real-time stream

First, we wanted to verify the presence of concept drift in the Avazu and Criteo datasets. To investigate it, we have used the ADWIN (ADaptive WINdowing) algorithm [11, 12], implemented via the Python `river` library. It expands the window when the data are stationary and abruptly shrinks it when a statistically significant change in the stream's mean is detected.

Experiments We have processed the chronological sequences of both the Avazu and Criteo streams, using the ADWIN estimator to continuously monitor the moving average of the target variable. The CTR and the detected drift points are presented in Figure 5 and Figure 6.

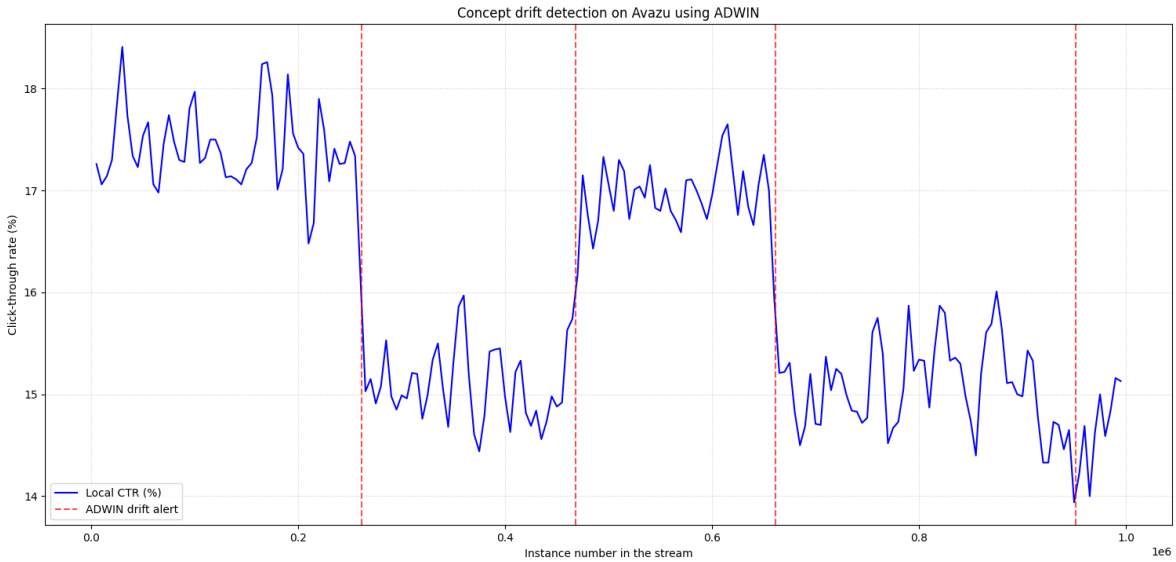


Figure 5: Concept drift detection using the ADWIN algorithm on the Avazu dataset.

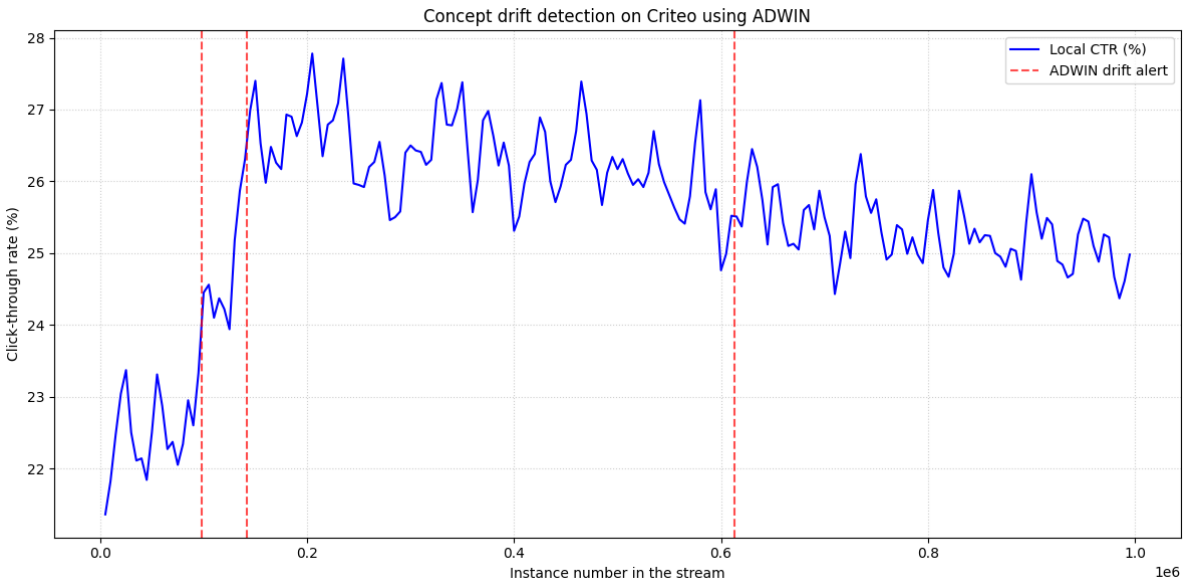


Figure 6: Concept drift detection using the ADWIN algorithm on the Criteo dataset.

As shown in the above plots, the CTR is highly volatile and not constant over time. For the Avazu stream, the local CTR fluctuates between 14% and 18.5%, triggering four different drift alerts over $1,000,000$ points. Similarly, the Criteo stream shows non-stationarity. CTR climbs from initial values around 21% up to almost 28%, setting off three main drift alerts.

As we can see, the ADWIN detector has identified critical points in both datasets where the probability distribution underwent major shifts. Hence, the presence of concept drift in these streams requires us to use dynamic online learning models to maintain predictive performance.

3.2 Evaluation procedure in the MOA environment

Due to the continuous nature of data streams, traditional machine learning evaluation techniques, such as cross-validation, are not applicable. We have used the prequential evaluation (test-then-train) method. In this approach, each incoming instance is first used to test the model – generating a prediction – and is then passed to the training algorithm to update the model's internal state. This means the classifier is always evaluated on unseen data.

Both real-world streams show class imbalance. Hence, we have abandoned the standard accuracy metric, which favors the majority class. Instead, we decided to evaluate our models using logarithmic loss (Log Loss) and area under the ROC curve (AUC ROC) [13]. The logarithmic loss heavily penalizes confident yet incorrect predictions, whereas the AUC-ROC evaluates the model's ability to correctly separate the positive (clicked) and negative (non-clicked) classes.

3.3 Establishing a baseline

We first established a baseline using the Hoeffding adaptive tree (HAT) – an algorithm for streaming classification. The baseline HAT model was evaluated on our processed Avazu and Criteo streams using the prequential procedure.

3.4 Adaptation gap on the synthetic stream

We have evaluated our baseline model on the synthetic Agrawal stream. We have injected an abrupt concept drift exactly at instance 500,000. Figure 7 illustrates the resulting adaptation gap.

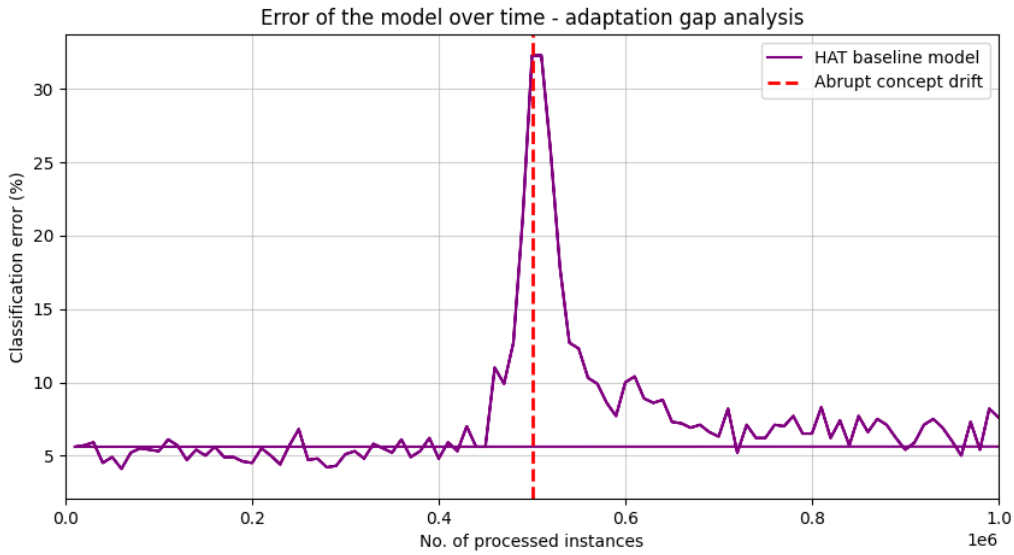


Figure 7: Error of the baseline HAT model over time, showing the adaptation gap after an abrupt concept drift.

During the initial stable phase, from 0 to 500,000 instances, the HAT has successfully learned the defined rules and has maintained a low predictive error. However, immediately upon drift injection, the tree structure has become obsolete, causing the classification error to spike to over 30%.

Furthermore, the recovery phase, in which the error slowly returns to a stable baseline, forms a distinct triangular shape on the plot. The width of it represents the recovery time ($t_{recovery}$). This leaves us with an important observation. Namely, standard tree adaptation is highly reactive and relatively slow. Therefore, flattening this exact error curve and minimizing $t_{recovery}$ are our primary motives for investigating the dynamic OFS as the next milestone.

4 Architecture of the proposed solution

4.1 Overall streaming pipeline

The implementation in the repository follows the standard streaming-learning loop: each instance is read in chronological order, transformed into a fixed feature representation, predicted by the current model, scored, and only then used for training. This protocol — known as prequential or test-then-train evaluation [3] — prevents information leakage from future observations into the current prediction and provides an unbiased estimate of the model's behaviour as a function of stream position.

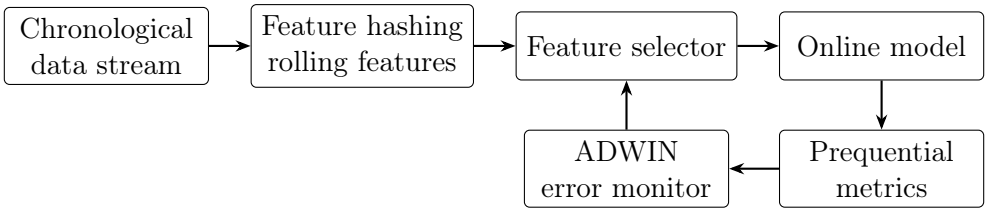


Figure 8: Architecture of the streaming evaluation and adaptation pipeline.

The Java implementation is organised around five groups of classes. Stream providers load either ARFF-based real datasets or the synthetic Agrawal stream through a unified `StreamProvider` interface. Model wrappers expose a common `StreamModel` contract for Hoeffding Tree, Hoeffding Adaptive Tree, SRP, and the proposed drift-aware SRP model. Feature selectors implement no selection, static top-K selection, periodic online ranking, and the drift-aware selection introduced in this work. The prequential evaluator coordinates prediction, metric updates, drift detection, model training, and CSV logging in a single integration loop. Decoupling the components through interfaces makes it possible to evaluate any combination of provider, selector and model without modifying the evaluator itself.

4.2 Baseline models and feature selectors

The baseline model set contains Hoeffding Tree (HT), Hoeffding Adaptive Tree (HAT), and Streaming Random Patches (SRP). HT is the basic incremental tree model for high-speed streams [14] and serves as a non-adaptive reference. HAT extends the tree with ADWIN-based monitors at internal nodes and replaces obsolete subtrees with alternative subtrees as the underlying concept changes. SRP is an ensemble method designed for evolving streams [5]; it combines online bagging with random feature subspaces and per-learner drift detection. In the repository, the SRP baseline uses 10 trees and a 60% random subspace setting, consistent with the default values used by the original authors.

Three baseline feature-selection strategies were implemented and evaluated:

- **No selector**: all attributes are passed to the model. This is the natural reference for measuring whether feature selection helps at all on a given stream.
- **Static top-K**: Information Gain is estimated on a warm-up segment of 5,000 instances and the selected features remain fixed for the rest of the stream.
- **Online ranking**: Information Gain is recomputed on a sliding window of 5,000 instances every 5,000 instances. The selected feature set therefore evolves with the stream, but only on a fixed schedule.

These baselines isolate the contribution of dynamic feature selection from the contribution of the classifier itself, and provide a comparison point for the proposed drift-aware variants.

4.3 Drift-aware feature selection

The proposed drift-aware selector replaces the periodic schedule of the online ranking baseline with an event-driven adaptation triggered by ADWIN. The key observation is

that the meaningful signal for feature selection is not the absolute Information Gain at a single point in time, but rather the change in Information Gain caused by the drift itself: features whose predictive value has dropped should be replaced precisely by features whose predictive value has risen.

The selector maintains a sliding buffer of recent instances. While no drift is signalled, the buffer simply rolls forward. When ADWIN raises a drift alert, the selector freezes the current buffer as the pre-drift snapshot and records the corresponding Information Gain values IG_{before}. It then continues to consume instances normally for one further window, accumulating post-drift data. Once the post-drift window is full, IG_{after} is computed and the per-feature change is taken as

$$\Delta IG_i \;=\; IG_{after,i} - IG_{before,i}.$$

Features for which ΔIG_i exceeds a positive threshold are treated as strengthened; features for which it falls below the symmetric negative threshold are treated as weakened. The new selected set is obtained from the current top-K ranking by adding strengthened candidates and removing weakened ones, with a length correction that brings the result back to size K if either operation overshoots.

For Avazu and Criteo, the selector keeps 20 features. For Agrawal, where the feature space is small, it keeps 5 features. Real CTR experiments use a 5,000-instance comparison window and a 0.005 Information Gain change threshold; Agrawal experiments use a smaller 800-instance window and a threshold of 0.05, reflecting the larger and more abrupt swings induced by the synthetic generator.

4.4 Drift-Aware SRP

The proposed model, Drift-Aware SRP (DASRP), extends the SRP framework along two complementary axes. First, ensemble subspaces become responsive to feature drift instead of being fixed at construction time. Second, members whose subspaces are reconstructed do not vote at full strength immediately: the ensemble keeps track of how much fresh evidence each member has seen and weighs its vote accordingly during recovery. Both mechanisms are designed to avoid the failure mode of a naive reset, in which the ensemble loses all accumulated knowledge on every drift event.

The implementation is split between two helper classes used by DASRP. The `SubspaceManager` owns the per-learner feature subsets and exposes an `adaptSubspaces` operation that consumes the weakened and strengthened feature sets produced by the drift-aware selector. For each learner, it counts the number of weakened features inside its subspace; learners below a configurable overlap threshold are left untouched, since their predictive basis was not affected by the drift. Learners above the threshold have their weak features replaced, preferring strengthened candidates and falling back to neutral features when the strengthened pool is too small. The fraction of the original subspace that had to be replaced is then compared against a reset ratio: subspaces that lost more than this fraction are flagged for full reinitialisation, since the surviving learner state would be inconsistent with its new feature set.

The `WeightManager` mirrors the reset decision on the voting side. Members reported by the subspace manager as reset have their weight reduced to a configurable value (0.3 in our experiments) and recover linearly toward the normal weight 1.0 at a fixed rate per

training instance. Aggregation is performed as a weighted average,

$$\hat{p} \;=\; \frac{\sum_{i=1}^{N} w_i\, p_i}{\sum_{i=1}^{N} w_i},$$

so that freshly reset members influence the ensemble in proportion to the amount of new data they have processed. With a recovery rate of 0.001 per instance, full recovery takes roughly 700 instances, which is comparable to the data volume needed by a Hoeffding tree to perform its first split.

The adaptation cycle ties everything together as follows:

1. ADWIN detects a change in the online error stream.
2. The drift-aware selector compares Information Gain before and after the drift and emits the weakened and strengthened feature sets.
3. The `SubspaceManager` removes weak features from affected subspaces and fills the gaps using strengthened features whenever possible.
4. Learners whose subspace changed beyond the reset ratio are reinitialised on the new sub-header.
5. The `WeightManager` reduces the vote weight of reset learners and gradually restores it during the subsequent training instances.

DASRP uses online bagging with $\lambda = 1$ rather than the $\lambda = 6$ value of the reference SRP implementation; this reduces the amplification of repeated training and keeps the focus of the experiments on the contribution of the drift-aware components. Drift detection is handled at the ensemble level by the external ADWIN monitor rather than by per-learner detectors, which keeps the adaptation policy concentrated in a single place.

On real CTR datasets, DASRP uses 10 base Hoeffding-tree learners, subspaces of size 20, ADWIN with $\delta = 0.002$, reset weight 0.3, recovery rate 0.001, and a reset ratio of 0.5. On Agrawal, the configuration uses subspaces of size 4, ADWIN with $\delta = 0.001$, reset weight 0.5, and recovery rate 0.005. The different settings reflect the much smaller feature space and sharper drift dynamics of the synthetic stream.

5 Final evaluation

5.1 Experiment matrix

The final evaluation used the CSV outputs from the implemented Java experiment pipeline. The baseline matrix contains 36 runs: 4 datasets, 3 models, and 3 feature selectors. The drift-aware matrix contains 12 additional runs: for each dataset, HT with the drift-aware selector, HAT with the drift-aware selector, and DASRP. In total, the report summarizes 48 prequential experiments.

Each run processes 200,000 instances. Metrics are logged every 1,000 instances. The main metrics are final Accuracy, final Log Loss, final AUC, and windowed 1,000-instance Log Loss. Log Loss is the most important metric for CTR prediction because the output probability is used directly in ranking and bidding decisions.

5.2 Real CTR results

Table 3: Final metrics on real CTR streams. Lower Log Loss is better.

Dataset	Method	Accuracy	Log Loss	AUC
Avazu	SRP + static top-K	0.8243	0.4659	0.7022
Avazu	**DASRP + drift-aware SRP**	**0.8253**	**0.4332**	**0.7048**
Criteo	SRP + online ranking	0.7557	0.5522	0.6512
Criteo	**DASRP + drift-aware SRP**	**0.7583**	**0.5291**	0.6511
Criteo	SRP + static top-K	**0.7634**	0.5622	**0.6754**

The most important result is that DASRP achieved the lowest final Log Loss on both real CTR datasets. On Avazu, DASRP improved Log Loss from 0.4659 for the best SRP baseline to 0.4332, which is a relative reduction of about 7.0%. On Criteo, DASRP improved Log Loss from 0.5522 to 0.5291, a relative reduction of about 4.2%. This supports the hypothesis that drift-aware feature adaptation is useful in high-dimensional CTR streams.

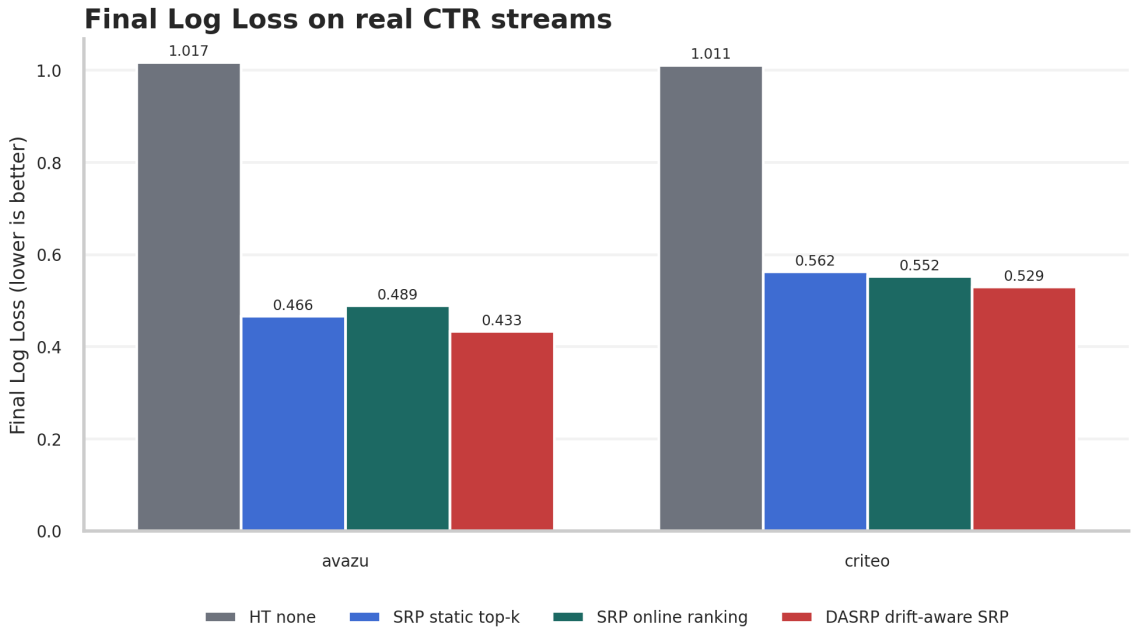


Figure 9: Final Log Loss comparison on Avazu and Criteo.

The time-series view in Figure 10 shows that the improvement is not only a final-point artifact. DASRP remains among the lowest-loss methods throughout the real CTR streams. The Avazu result is particularly stable. On Criteo, static SRP is competitive early but degrades later, while DASRP avoids the strongest late loss increase.

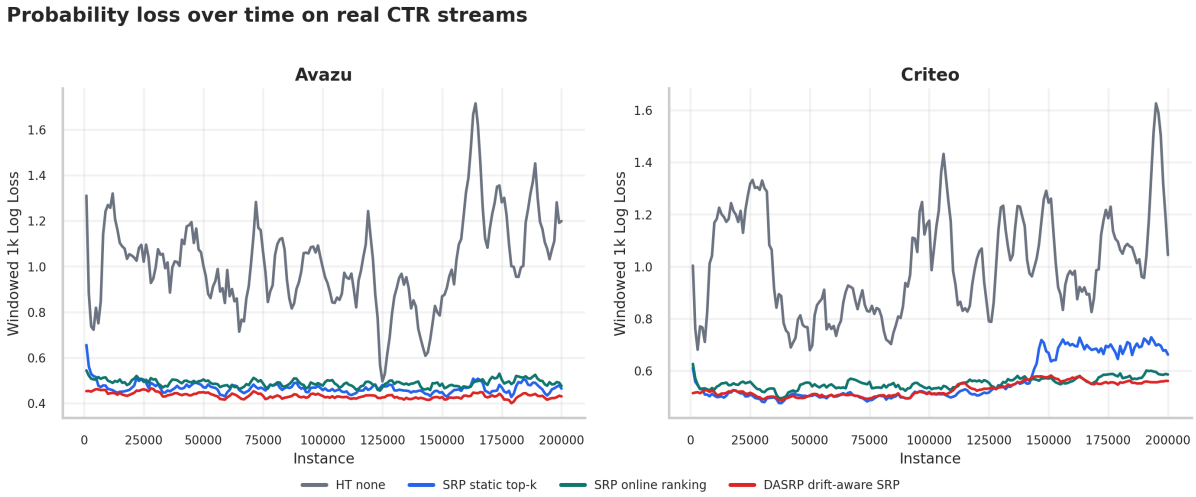


Figure 10: Windowed 1,000-instance Log Loss over time on real CTR streams.

5.3 Runtime analysis

Runtime is a key constraint in streaming systems. A model that provides good Log Loss but cannot process the stream fast enough is not practical. DASRP was much faster than the strongest SRP baselines on real CTR streams because it adapts only selected ensemble members instead of rebuilding or maintaining a large full SRP configuration.

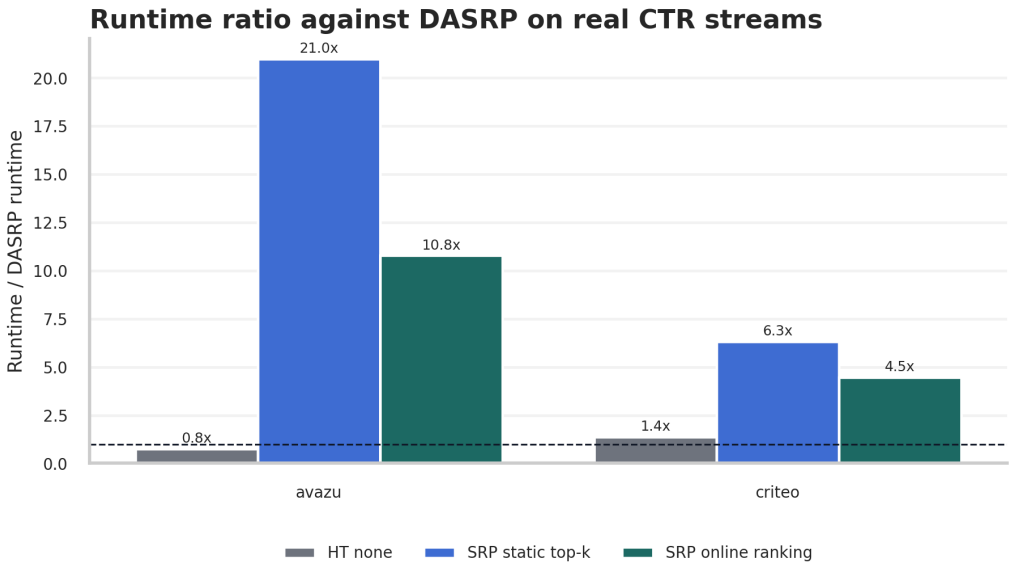


Figure 11: Runtime ratio of selected real-stream baselines against DASRP. Values above 1 mean the baseline was slower than DASRP.

On Avazu, SRP with static top-K took approximately 21 times the DASRP runtime, and SRP with online ranking took approximately 10.8 times the DASRP runtime. HT was faster than DASRP, but it was much worse in final Log Loss. This indicates that DASRP is not the fastest method overall, but it provides an attractive quality/runtime trade-off among the competitive ensemble methods.

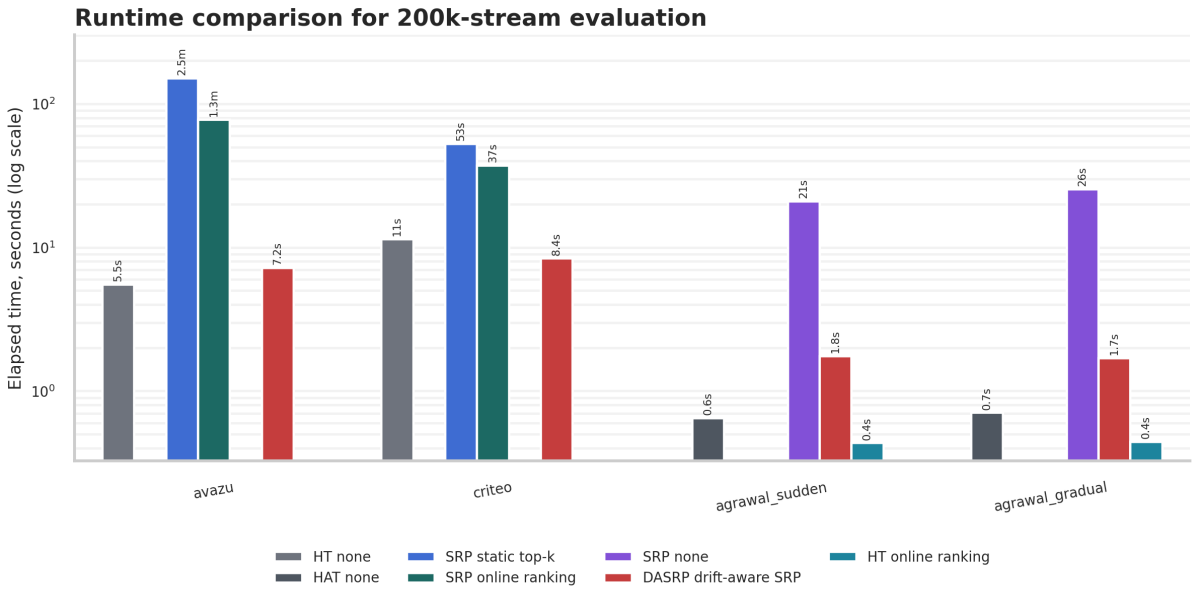


Figure 12: Absolute runtime comparison for 200,000-instance runs. The vertical axis is logarithmic.

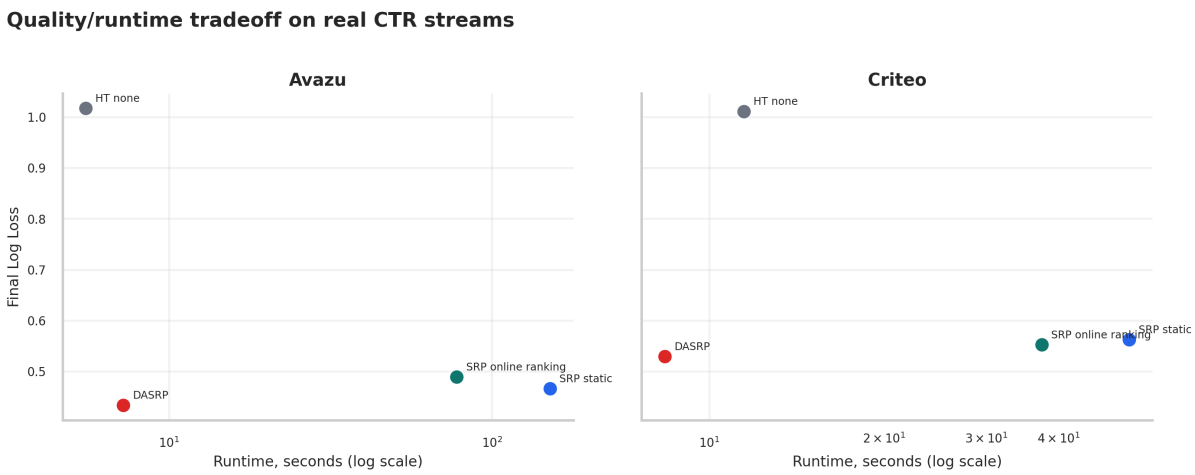


Figure 13: Quality/runtime trade-off on real CTR streams. Lower and further left is better.

5.4 Synthetic Agrawal results

Table 4: Final metrics on controlled Agrawal drift streams.

Dataset	Method	Accuracy	Log Loss	AUC
Sudden Agrawal	HAT + no selector	**0.9559**	**0.1604**	0.9966
Sudden Agrawal	HT + online ranking	0.9279	0.1939	**0.9982**
Sudden Agrawal	DASRP + drift-aware SRP	0.8435	0.4553	0.8838
Gradual Agrawal	HT + online ranking	0.9195	**0.2171**	0.9979
Gradual Agrawal	HAT + no selector	**0.9282**	0.2420	0.9975
Gradual Agrawal	DASRP + drift-aware SRP	0.8029	0.4848	0.9067

The synthetic results are less favorable for DASRP. In the low-dimensional Agrawal setting, HAT and periodic online ranking recover quickly after the controlled drift. DASRP is more conservative and does not match the best synthetic baselines. This suggests that the method is most useful when the stream is high-dimensional and feature relevance changes are meaningful.

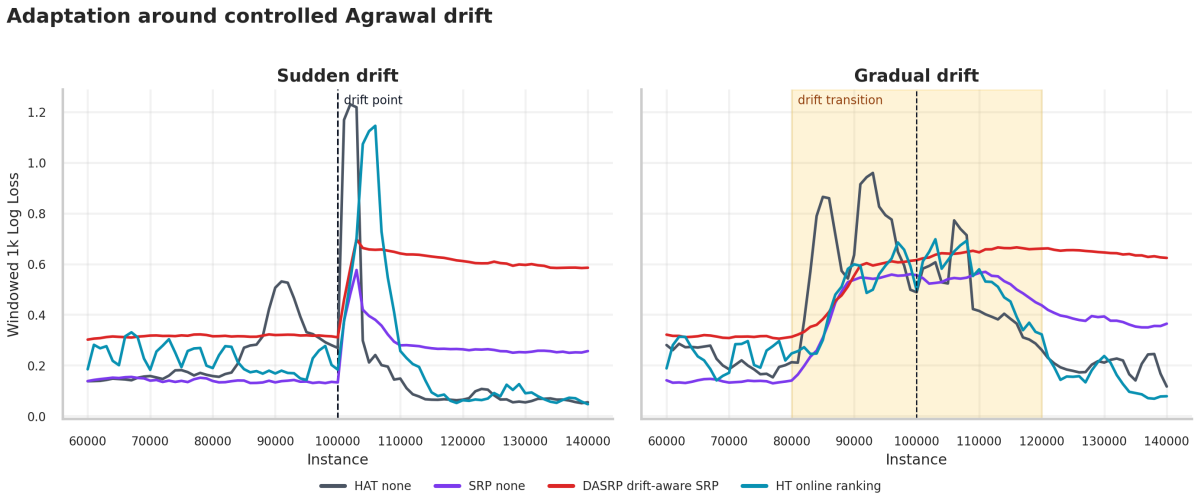


Figure 14: Windowed Log Loss around controlled Agrawal drift.

The number of ADWIN alarms also shows that drift-aware mechanisms require careful tuning. In simple synthetic streams, frequent alarms may lead to unnecessary feature and model changes.

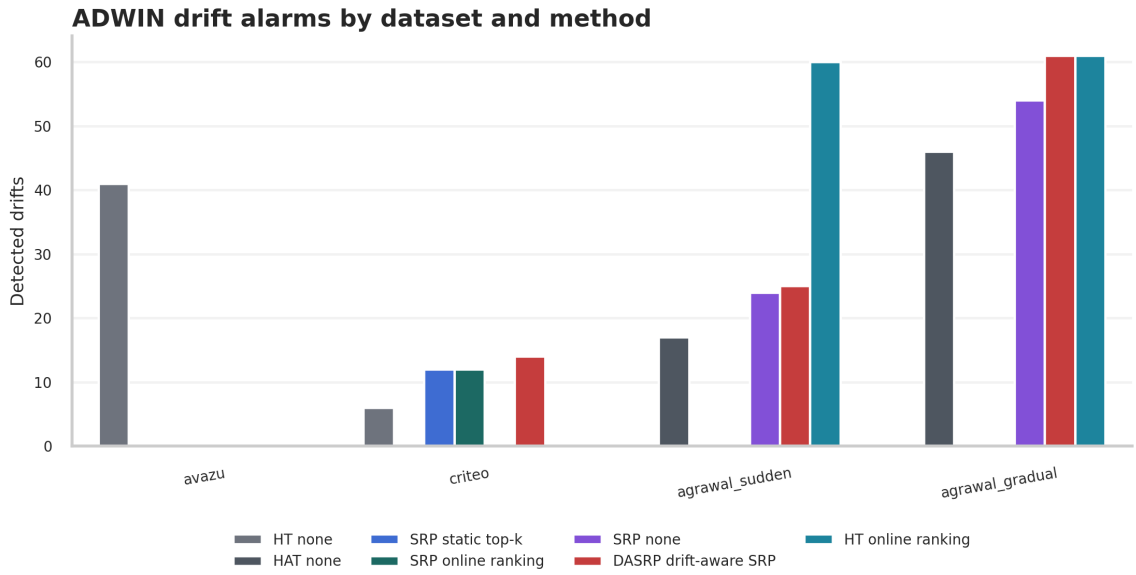


Figure 15: ADWIN drift alarms by dataset and selected method.

6 Conclusion

The final experiments confirm that online feature adaptation can improve CTR prediction in high-dimensional, non-stationary streams. On Avazu and Criteo, DASRP achieved the best final Log Loss among the evaluated methods and was substantially faster than the strongest SRP baselines. This is the most important outcome of the project because CTR systems rely on calibrated probabilities rather than only hard class labels.

The synthetic experiments provide an important limitation. On Agrawal streams, standard adaptive trees and periodic online ranking already recover very well, while DASRP underperforms. This means that the proposed architecture should not be treated as universally superior. It is most appropriate when the feature space is large, sparse, and unstable.

Future work should include full million-instance experiments, a formal recovery-time metric around drift events, and a more systematic search over ADWIN sensitivity, Information Gain thresholds, subspace sizes, reset ratio, and weight recovery rate.

7 Team contributions

Table 5: Division of work across team members, including detailed contributions and time assessment.

Name	Contributions	Workload
Hubert Jaczyński	**Analytics and final evaluation:** concept drift detection in raw data with ADWIN; final evaluation and comparative analysis of the proposed algorithm against baselines. Fixes to method implementation. **Report:** final evaluation and conclusion sections. **Presentation creation.**	∼32h
Aleksandra Kłos	**Data engineering and baseline evaluation:** preprocessing Avazu and Criteo streams, exploratory data analysis, feature hashing, missing-value imputation, baseline evaluation in MOA, and scripts for Log Loss and adaptation-gap analysis. **Report:** introduction, data preparation, and evaluation methodology sections. **Presentation creation.**	∼32h
Jakub Oganowski	**Algorithm implementation:** Java implementation of the SRP/OFS integration, drift-aware feature selection, subspace adaptation, and ensemble weighting. **Report:** architecture of the proposed solution section. **Presentation creation.**	∼32h

References

[1] H. Brendan McMahan et al. "Ad click prediction: a view from the trenches". In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2013, pp. 1222–1230.

[2] Pedro Domingos and Geoff Hulten. "Mining high-speed data streams". In: *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2000, pp. 71–80.

[3] Joao Gama et al. "A survey on concept drift adaptation". In: *ACM Computing Surveys* 46.4 (2014), pp. 1–37.

[4] Haiguang Li et al. "Group feature selection with streaming features". In: *2013 IEEE 13th International Conference on Data Mining*. 2013, pp. 1109–1114.

[5] Heitor Murilo Gomes, Jesse Read, and Albert Bifet. "Streaming random patches for evolving data stream classification". In: *2019 IEEE International Conference on Data Mining*. 2019, pp. 240–249.

[6] Congcong Liu et al. "On the adaptation to concept drift for CTR prediction". In: *arXiv preprint arXiv:2204.05101* (2022).

[7] Vladimir Vovk. "The fundamental nature of the log loss function". In: *Fields of Logic and Computation II*. Springer, 2015, pp. 307–318.

[8] Albert Bifet et al. "MOA: Massive online analysis, a framework for stream classification and clustering". In: *Proceedings of the First Workshop on Applications of Pattern Analysis*. 2010, pp. 44–50.

[9] Kilian Weinberger et al. "Feature hashing for large scale multitask learning". In: *Proceedings of the 26th Annual International Conference on Machine Learning*. 2009, pp. 1113–1120.

[10] Rakesh Agrawal, Tomasz Imielinski, and Arun Swami. "Database mining: A performance perspective". In: *IEEE Transactions on Knowledge and Data Engineering* 5.6 (1993), pp. 914–925.

[11] Albert Bifet and Ricard Gavalda. "Learning from time-changing data with adaptive windowing". In: *Proceedings of the 2007 SIAM International Conference on Data Mining*. 2007, pp. 443–448.

[12] Albert Bifet and Ricard Gavalda. "Adaptive learning from evolving data streams". In: *International Symposium on Intelligent Data Analysis*. 2009, pp. 249–260.

[13] Tom Fawcett. "An introduction to ROC analysis". In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874.

[14] Geoff Hulten, Laurie Spencer, and Pedro Domingos. "Mining time-changing data streams". In: *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2001, pp. 97–106.